

改进后的完整软件过程定义文档

项目名称：校园活动/社群聚合与匹配平台

项目描述：本项目是一个面向校内学生的轻量级 Web 应用，主要功能包括活动信息发布、组队申请提交、申请审核与权限管理。项目团队由 4 人组成，开发周期约 8 周，采用迭代式开发模式。

版本：2.0 (AI增强版)

编制人：曾琪

日期：2026年6月

本文档是在原轻量级软件过程基础上，融入AI技术后的完整版本。原有核心框架（ISO/IEC 12207剪裁、敏捷迭代、MVC架构）保持不变，新增内容已整合至各章节。

一、过程概述与总体目标

1.1 过程背景

本过程定义适用于“校园活动/社群聚合与匹配平台”项目。该项目是一个面向校内学生的轻量级Web应用，主要功能包括活动信息发布、组队申请提交、申请审核与权限管理。开发团队共4人，采用迭代式开发模式，每个迭代周期为两周。技术栈为Java Servlet + JSP + JDBC + MySQL，采用B/S三层架构和MVC设计模式。

本过程依据ISO/IEC 12207:2017标准框架，结合敏捷思想进行剪裁，保留了需求分析、架构设计、编码实现、集成测试、部署交付等核心技术活动，简化了文档和正式评审要求。同时，为提升效率和质量，本过程集成了生成式AI工具（如大语言模型、GitHub Copilot、SonarQube AI插件等），并在关键环节设置了AI辅助节点。

1.2 过程总体目标

- 交付效率：**每个迭代周期末交付可运行的产品增量，需求整理、编码、测试等核心活动的耗时较传统人工方式显著降低。
- 产品质量：**系统核心功能的缺陷密度控制在每千行1.5以下，测试分支覆盖率达到85%以上。
- 过程可控：**所有AI辅助环节均留有日志和评审记录，AI输出物需经人工确认，确保人机协同不失控。
- 持续改进：**通过收集AI贡献度指标和失效案例，定期优化提示词模板和 workflows。

1.3 过程剪裁原则

- 保留技术过程（需求、设计、编码、测试、部署）的全链条，确保工程完整性。
- 保留技术管理过程（策划、质量保证、配置管理），采用非正式评审和每日站会代替正式会议。
- 新增轻量级交付物（AI使用日志、提示词模板库、AI失效案例库），不增加额外管理负担。
- 省略长期维护过程和独立测试团队。

二、各阶段活动流程图（含AI参与节点）

下图展示了本过程的完整活动流，橙色节点表示AI参与的活动，蓝色节点表示人工主导的活动。

需求阶段

用户访谈

AI辅助生成用户故事

团队评审与修订

需求基线确认

设计阶段

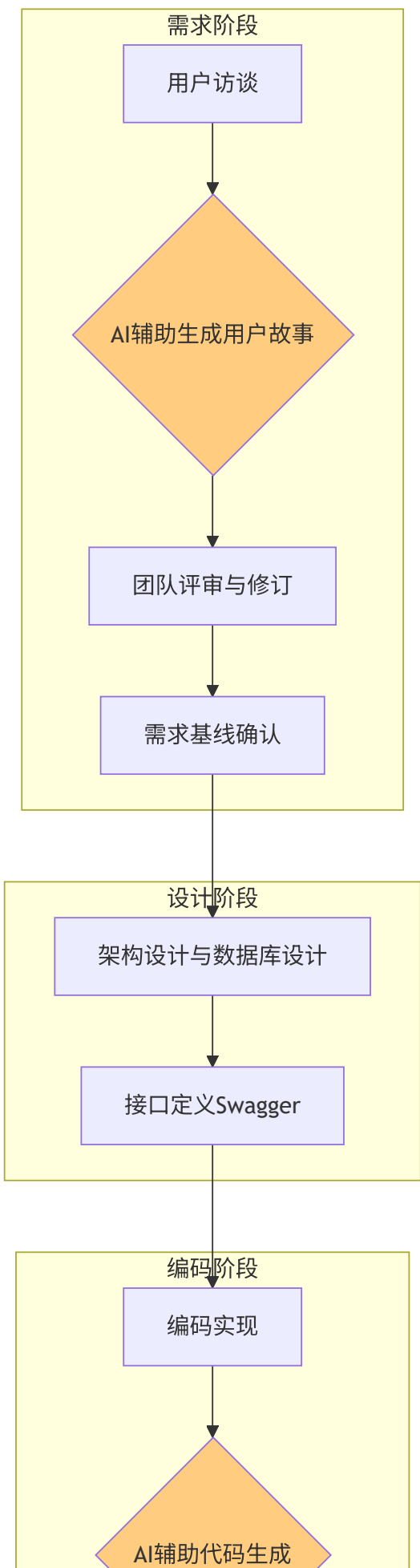
架构设计与数据库设计

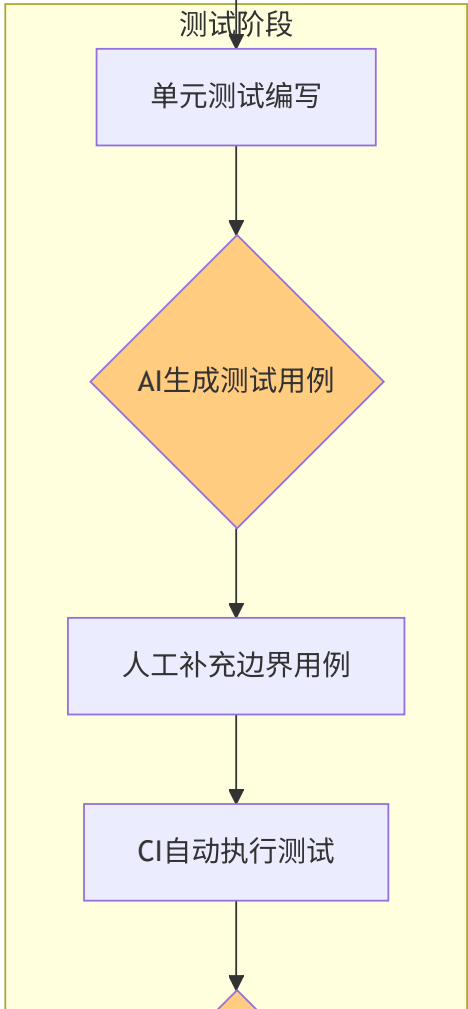
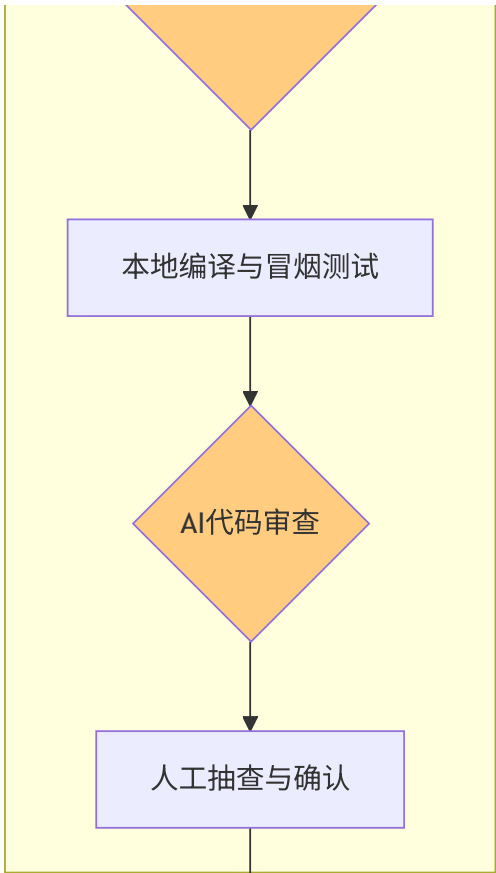
接口定义Swagger

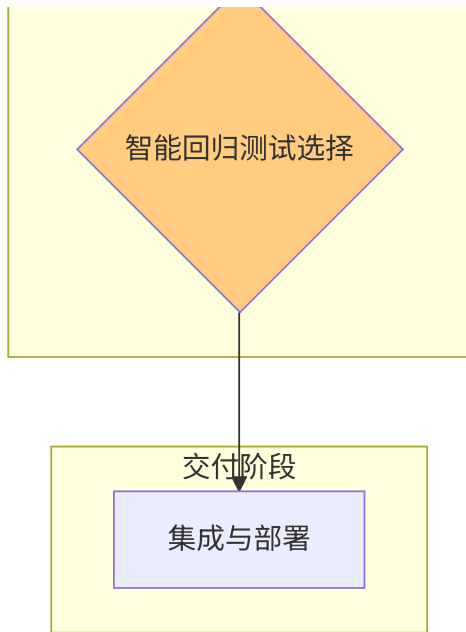
编码阶段

编码实现

AI辅助代码生成







流程说明：

- 需求阶段：AI根据访谈纪要生成用户故事草稿及验收标准，团队评审后确认基线。
- 设计阶段：人工完成架构和数据库设计，输出Swagger接口定义。
- 编码阶段：AI辅助实时生成代码（基于方法签名和注释），AI静态分析检测缺陷，人工抽查业务逻辑。
- 测试阶段：AI生成基础测试用例，人工补充边界，AI智能选择回归测试子集。
- 各AI节点后均设有强制的人工评审或修正环节，确保质量可控。

三、角色职责定义

3.1 项目经理（兼任AI协调员）

主要职责：

- 制定迭代计划、跟踪进度、协调资源、管理风险。
- 维护AI提示词模板库（需求生成、代码生成、测试生成三类模板），存储于 `/docs/ai-templates`。
- 记录每次AI调用的元数据（时间、模型、提示词摘要、输出摘要、是否修正），存入 `/ai-logs`。
- 组织每周一次的AI输出物评审会议（30分钟），评估效率与质量，调整提示词策略。
- 管理AI工具的账号和成本预算。

3.2 后端开发

主要职责：

- 编写Servlet、DAO、Service、数据库操作。
- 使用GitHub Copilot辅助编码，在提交注释中标注“AI:copilot”。
- 本地编译并执行冒烟测试，确保AI生成代码无语法错误且逻辑正确。
- 参与代码审查，重点关注AI生成部分的业务语义。

3.3 前端开发

主要职责：

- 编写JSP、HTML/CSS/JS，与后端联调。
- 可尝试使用AI生成前端代码（非强制）。

3.4 测试人员（兼任验证分析师）

主要职责：

- 编写测试用例、执行测试、报告缺陷、回归验证。
- 制定AI生成测试用例的验收标准（分支覆盖率≥85%，首次通过率≥90%）。
- 每周随机抽取20%的AI生成测试用例进行人工审查，记录无效断言或遗漏场景。
- 处理CI流水线中AI生成测试的失败用例，判别根因（AI误判/代码缺陷/环境问题）。
- 维护“AI失效案例库”(/ai-logs/failures.md)，记录模型频繁出错的模式，用于优化提示词。

3.5 角色协作说明

- 项目经理（AI协调员）提供提示词模板和AI日志，组织每周评审。
- 开发人员使用AI辅助编码，测试人员（验证分析师）审查AI生成的测试用例并反馈失效案例。
- 每周评审会议中，全体角色共同讨论AI使用效果和改进措施。

四、交付物清单与质量标准

4.1 需求阶段交付物

交付物	格式	质量标准	生成方式
用户故事列表	Markdown表格	每个故事包含正常及异常验收标准； 团队评审驳回率≤20%	AI生成初稿， 人工修订
需求追溯矩阵	Excel/CSV	测试用例与需求双向追溯，遗漏率≤10%	人工建立

4.2 设计阶段交付物

交付物	格式	质量标准
数据库ER图	图片或SQL脚本	满足第三范式，与需求一致
Swagger/OpenAPI文档	YAML/JSON	覆盖所有核心接口，可导出为文档

4.3 编码阶段交付物

交付物	格式	质量标准	生成方式
源代码 (Java/JSP)	.java, .jsp	通过SonarQube零严重警告；缺陷密度≤1.5/千行； AI生成代码标注“AI:copilot”	AI生成方法体， 人工审查
代码审查记录	PR评论	每周抽查20%的AI生成代码，确认业务逻辑	人工审查

4.4 测试阶段交付物

交付物	格式	质量标准	生成方式
JUnit测试类	.java	分支覆盖率≥85%；AI生成用例首次通过率 ≥90%	AI生成骨架， 人工补充边界
CI测试报告	HTML/JSON	包含失败用例根因分类；回归执行时间 ≤60分钟	人工分析， AI辅助回归选择
AI失效案例库	Markdown	每周更新，记录案例及改进措施	验证分析师维护

4.5 过程日志交付物

交付物	格式	质量标准
AI使用日志	CSV	完整性≥95%，修正原因字段填写率100%
提示词模板库	.txt/.md	每个模板包含角色、任务、约束、输出格式示例，经团队评审
每周AI评审会议记录	Markdown	包含效率数据、质量问题、改进计划

五、过程度量指标（含AI贡献度指标）

5.1 效率指标

指标	基线（无AI）	目标（有AI）	测量方法	数据来源
需求整理耗时（人天/迭代）	2	0.5	Trello任务卡片	工时记录
CRUD编码耗时（人天/模块）	1.5	0.7	Git log + 工时记录	版本控制
测试编写时间（分钟/接口）	55	25	个人计时+审查记录	审查日志
回归测试执行时间（分钟）	120	≤60	CI流水线日志	GitHub Actions

5.2 质量指标

指标	基线（无AI）	目标（有AI）	测量方法	数据来源
需求遗漏率	12%	≤8%	需求追溯矩阵	测试与需求对比
测试分支覆盖率	65%	≥85%	JaCoCo报告	CI报告
缺陷密度（缺陷/千行）	2.2	≤1.5	缺陷管理系统	Trello缺陷卡片
AI输出人工评审通过率	—	≥90%	每周抽查记录	审查日志

5.3 AI贡献度指标

指标	定义	测量方法	目标值
AI代码生成比例	AI生成代码行数 / 总代码行数	提交注释中的 “AI:copilot” 标签统计	≤40%
AI用例首次通过率	无需修正即可编译通过并正确断言的AI生成测试用例比例	审查记录	≥90%
AI失效案例数	记录在案的AI输出错误或幻觉案例	失效案例库条目数	每周 ≤2个
提示词复用次数	同一提示词模板被使用的次数	AI调用日志	每模板 ≥5次
AI调用成本	调用AI服务产生的费用（元/迭代）	API账单或手动记录	≤50元/ 迭代

5.4 度量数据收集方式

- 工时数据：Trello任务卡片记录开始和结束时间。
- 代码度量：SonarQube每周扫描生成报告。
- 测试覆盖率：JaCoCo Maven插件在CI中运行。
- AI日志：调用脚本自动记录请求和响应元数据，人工补充修正信息。
- 缺陷数据：从Trello缺陷卡片统计，按严重级别分类。

六、过程剪裁说明

本过程基于ISO/IEC 12207剪裁，保留了核心工程活动，同时融入了AI工具支持。剪裁决策如下：

活动或文档	本过程做法	理由
软件需求规格说明书	使用用户故事+功能列表替代	项目规模小，轻量化沟通更高效
独立测试团队	开发人员自测+交叉测试	团队人力有限，测试与开发紧密协作
详细设计说明书	代码注释+类图说明	维护成本高，代码即设计
正式技术评审会议	日常沟通+代码走查	团队面对面协作频繁，非正式更灵活
风险分析文档	计划阶段口头讨论	风险可控，无需独立文档
长期维护过程	省略	课程项目无需运维
用户故事编写	AI生成初稿，人工修订	减少重复劳动，提高效率
CRUD代码编写	AI自动生成，人工审查	大量重复代码，AI效率高且不易出错
测试用例编写	AI生成骨架，人工补充边界	AI覆盖常见场景，人工专注边界
AI使用过程	记录日志、维护失效案例库	用于过程改进和问题回溯

七、总结

本过程定义文档（版本2.0）在保持原有轻量级敏捷框架的基础上，系统性地融入了AI技术支持。通过明确各角色职责（其中项目经理兼任AI协调员、测试人员兼任验证分析师）、定义AI输出物质量标准、设立AI贡献度指标、增加轻量级日志和失效案例库，实现了效率与质量的同步提升。本过程已通过测试用例生成场景的最小可行验证，预期可在后续迭代中全面实施。项目经理（AI协调员）将每周收集度量数据，定期评审过程执行情况，持续优化提示词模板和质量门限。

文档结束